NuggetIndex: Governed Atomic Retrieval for Maintainable RAG

Saber Zerhoudi
University of Passau
Passau, Germany
szerhoudi@acm.org

Michael Granitzer
Interdisciplinary Transformation
University Austria
Linz, Austria
University of Passau
Passau, Germany
michael.granitzer@uni-passau.de

Jelena Mitrović
University of Passau
Passau, Germany
jelena.mitrovic@uni-passau.de

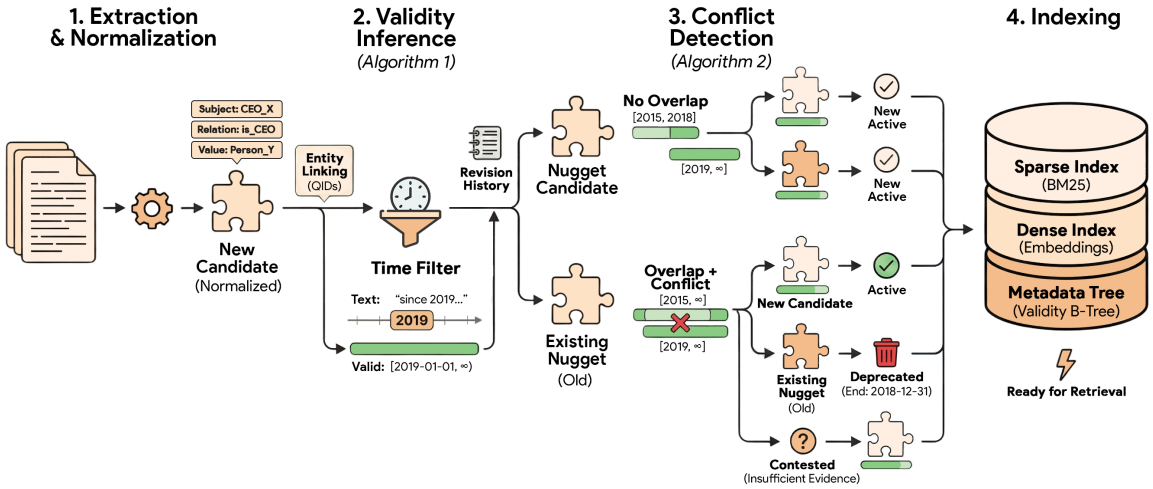


Figure 1: NuggetIndex pipeline. Raw text is normalized into atomic candidates. Algorithm 1 infers validity intervals using temporal expressions and revision history, while Algorithm 2 detects conflicts with the index to determine lifecycle states.

Abstract

Retrieval-augmented generation (RAG) systems are frequently evaluated via fact-based metrics, yet standard implementations retrieve passages or static propositions. This unit mismatch between evaluation and retrieval objects hinders maintenance when corpora evolve and fails to capture superseded facts or source disagreements. We propose NuggetIndex, a retrieval system that stores atomic information units as managed records, so called nuggets. Each record maintains links to evidence, a temporal validity interval, and a lifecycle state. By filtering invalid or deprecated nuggets prior to ranking, the system prevents the inclusion of outdated information. We evaluate the approach using a nuggetized MS MARCO subset, a temporal Wikipedia QA dataset, and a multi-hop QA task. Against passage and unmanaged proposition retrieval baselines, NuggetIndex improves nugget recall by 42%, increases temporal correctness by 9 percentage points without the recall collapse observed in time-filtered baselines, and reduces conflict rates by 55%. The compact nugget format reduces generator input length by 64% while enabling lightweight index structures suitable for browser-based and resource-constrained deployment. We release our implementation, datasets, and evaluation scripts[1].

CCS Concepts

• **Information systems** → **Retrieval models and ranking**; *Users and interactive retrieval*; **Web searching and information discovery**; *Evaluation of retrieval results*; • **Computing methodologies** → **Natural language processing**.

Keywords

Retrieval-augmented generation, Information retrieval, Nugget-based evaluation, Large language models, Agentic systems

ACM Reference Format:
Saber Zerhoudi, Michael Granitzer, and Jelena Mitrović. 2026. NuggetIndex: Governed Atomic Retrieval for Maintainable RAG. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '26), July 20–24, 2026, Melbourne, VIC, Australia.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3805712.3809687

1 Introduction

Retrieval-augmented generation (RAG) grounds large language model (LLM) outputs in external evidence to reduce hallucination [24]. A primary challenge for deployed RAG systems is maintaining correctness as the underlying corpus evolves [17, 32].

[1]https://github.com/searchsim-org/sigir26-nuggetindex

This work is licensed under a Creative Commons Attribution 4.0 International License.
SIGIR '26, Melbourne, VIC, Australia
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2599-9/2026/07
https://doi.org/10.1145/3805712.3809687

In dynamic domains such as software documentation, news, or law, updates occur frequently [12]. Standard passage-based systems may retrieve conflicting information from different versions of a document. The generator might then merge outdated and current details, producing confident but incorrect answers. This issue arises from retrieving valid-looking but temporally inconsistent evidence.

Recent research suggests that retrieving finer-grained units, such as propositions or triplets, improves relevance and reduces token costs [11], and that augmenting RAG with user-centric agents yields further gains on personalised tasks [47]. However, existing implementations store these units as static text or embeddings, lacking metadata to capture temporal validity or source disagreement.

Simultaneously, evaluation methodologies are shifting toward nugget-based scoring, where a "nugget" is defined as an atomic, self-contained fact [34, 41]. This trend creates a design misalignment: systems are evaluated on precise atomic facts but retrieve unmanaged, multi-fact passages. Consequently, retrieval systems lack mechanisms to explicitly track or update the specific facts required for accurate generation.

We propose bridging this gap with NuggetIndex, an AI-governed index where retrieval units are not merely strings, but **versioned records with epistemic annotations**. We formally define a nugget not as a text span, but as a composite record $N = (f, v, e)$, containing the atomic fact f, its temporal validity v, and its epistemic status e (e.g., agreed vs. contested).

This data model relies on three properties to ensure maintenance. First, every nugget is assigned a *temporal validity interval* [37], $[t_s, t_e)$, defining when the fact is true, rather than when it was written. Second, we explicitly model epistemic uncertainty through a lifecycle state: Active for current consensus, Deprecated for superseded facts, and Contested for unresolved disagreement. Third, nuggets maintain provenance links to source spans, grounding these validity judgments in evidence.

At query time, NuggetIndex filters the index to retain only nuggets that are Active and valid for the specific query time. Deprecated nuggets are excluded, while Contested items can be explicitly flagged. This filtering occurs before ranking, ensuring the generator processes only consistent and valid evidence.

We make the following contributions:

(1) We formalize RAG maintenance as a retrieval task and identify the discrepancy between nugget evaluation and retrieval.
(2) We introduce the NuggetIndex data model, which integrates validity intervals and lifecycle states, alongside algorithms for index construction and conflict resolution.
(3) We propose a governed retrieval pipeline that filters by temporal validity and lifecycle states before ranking. Ablations reveal that sparse retrieval outperforms dense on atomic nuggets, enabling sub-millisecond latency without embedding inference.
(4) We evaluate NuggetIndex across four benchmarks, demonstrating a 42% increase in nugget recall, 9.1 percentage point improvement in temporal correctness, and 55% reduction in conflict rates, while reducing generator input by 64%.

2 Background and Related Work

This section reviews four research areas relevant to our study: retrieval granularity, nugget-based evaluation, temporal information retrieval, and structured knowledge. We position NuggetIndex relative to these approaches at the end of the section.

2.1 Retrieval Granularity

Standard RAG systems typically retrieve passages ranging from 100 to 200 tokens and concatenate them to form a prompt for the generator [18, 19, 24]. While this approach is straightforward, it suffers from inherent limitations. Passages frequently contain a mixture of relevant and irrelevant content, and multiple passages may redundantly state the same information. Furthermore, long contexts increase the computational cost of generation [26].

To address these issues, recent research has explored retrieving finer-grained units. For instance, Chen et al. [11] extract propositions—single-sentence atomic statements—and perform retrieval at this level, reporting improved precision. Similarly, triplet-based approaches extract subject-relation-object tuples and retrieve over these structures [16]. These methods succeed in improving relevance density and reducing prompt length. However, such atomic units are generally stored as plain text strings or embeddings without additional structure. They lack metadata indicating when a fact is valid or whether it has been superseded. Consequently, if two propositions contradict one another, the system lacks the mechanism to detect or resolve the conflict, potentially retrieving both and passing the contradiction to the generator.

2.2 Nugget-Based Evaluation

Nugget-based evaluation originated in the TREC QA tracks, where system answers were scored based on their coverage of "information nuggets"—atomic facts that a complete answer should contain [33, 42]. This method offers greater robustness than string matching because it credits semantically equivalent answers even when their surface forms differ.

Recent advancements have scaled nugget-based evaluation through the use of LLM-based autograders [34]. Given a candidate answer and a set of gold nuggets, an autograder determines which nuggets the answer successfully covers, enabling evaluation on large benchmarks without requiring manual annotation for every response. The reliance on nugget coverage offers an opportunity to align retrieval design with evaluation by targeting nuggets directly. This alignment aids in diagnosing failures and enables the optimization of retrieval models for nugget recall.

A parallel line of work uses nugget-like atomic units inside the retrieval pipeline rather than only at evaluation time. GINGER [22] grounds response generation in atomic information nuggets retrieved per query; proposition-style retrievers [11] index single-sentence propositions and report precision gains on factoid QA; triplet-driven retrievers [16] index subject–relation–object tuples for RAG. These approaches share our atomic-unit choice but treat units as static text, omitting the validity interval and lifecycle state we maintain. On the temporal knowledge graph side, hybrid structural–semantic frameworks [14] model the evolution of relations between entities over time and answer queries against curated graphs that combine structural and semantic signals to capture historical patterns; these systems rely on a fixed schema and pre-existing graph structure. NuggetIndex sits between these two lines: we adopt the atomic-unit retrieval substrate of nugget-IR

Table 1: Comparison of retrieval approaches for RAG.

	Gran.	Temporal		Baseline	Eff.
Method	AU	VI	LS	IR	Cost
Passage RAG	–	–	–	✓	✓
Proposition RAG	✓	–	–	✓	✓
Wikidata + retrieval	✓	✓	✓	–	–
GraphRAG	–	–	–	–	–
NuggetIndex	✓	✓	✓	✓	✓

Gran. granularity; Eff. efficiency. AU atomic units; VI validity intervals; LS lifecycle states; IR standard IR; Cost low query cost. ✓ = supported; – = not supported.

while importing the validity-interval discipline of temporal KGs into the retrieval index itself, with the schema discovered from the target corpus rather than authored in advance.

2.3 Temporal Information Retrieval

Temporal Information Retrieval (IR) incorporates time into retrieval processes. Document-level methods use publication timestamps to rank by recency or filter within time windows [25]. Query-level approaches identify temporal intent to adjust retrieval strategies [6].

In the context of RAG, the core challenge is *fact-level* temporal validity. A document published in 2023 may discuss events from 2020, while a document from 2018 may state a fact that remains true until 2022. Document timestamps do not capture these nuances; validity information is required at the fact level. Temporal question answering benchmarks evaluate whether systems provide answers appropriate for a specific query time [12]. Systems that retrieve mixed-time evidence typically perform poorly on these evaluations.

2.4 Structured Knowledge for RAG

Knowledge bases like Wikidata store facts with qualifiers such as temporal constraints and references [1, 43]. Wikidata statements can also be marked with ranks, such as preferred or deprecated, to indicate currency. Although this model is conceptually close to our proposed approach, Wikidata relies on a fixed ontology and formal schema. Populating the base requires entity linking and relation extraction against that specific schema, and querying requires structured languages like SPARQL [2]. These requirements limit the applicability of such systems to open-domain text corpora where facts do not conform to a predefined structure.

Alternatively, GraphRAG approaches construct graphs over documents and utilize graph traversal or graph neural networks for evidence retrieval [20]. Although these methods enable multi-hop reasoning by capturing relationships, they require expensive graph construction and suffer from increased inference latency. Industrial systems such as AWS Neptune's Lexical Graph [3] hybridize lexical and graph indices to reduce that cost, but, like GraphRAG, they treat retrieved evidence as static and surface no temporal-validity, conflict, or rename signals to the generator.

2.5 Positioning of NuggetIndex

Table 1 compares NuggetIndex to prior approaches, with the main distinction being the integration of validity data.

vs. GraphRAG: GraphRAG [20] emphasizes structural connectivity and multi-hop traversals. These implementations typically treat edges as static, lacking mechanisms to track temporal status or disputes. NuggetIndex prioritizes *validity governance* over topological complexity, ensuring retrieved facts are temporally accurate.

vs. Wikidata: Wikidata models validity using a ranking system (Preferred, Normal, Deprecated) [43]. This relies on manual curation and rigid ontologies. NuggetIndex approximates this model via automated extraction from unstructured text, creating a scalable, *probabilistic Wikidata* that operates without human maintenance.

In summary, NuggetIndex defines the *validity-scoped fact* as the fundamental unit of automated retrieval.

3 NuggetIndex: Governed Nugget Retrieval

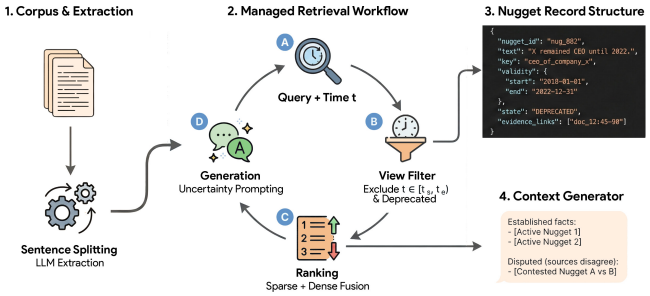


Figure 2: The NuggetIndex architecture.

Documents are processed into atomic nuggets with temporal validity intervals and lifecycle states. At query time, the system filters by validity and state before ranking.

Operational scope and terminology. We use *fact* to denote a (subject, predicate, object) triple extracted from a document span; we do not address philosophical truth or epistemological certainty. *Validity* is the time interval during which a source asserts the fact, recovered from explicit temporal expressions when present (Algorithm 1); when absent, the interval is anchored to the document timestamp and flagged as inferred. *Consensus* is operationalized as agreement of ≥ 2 independent sources on the same canonicalized key; it is corpus-relative, not absolute. A fact is Contested when two or more sources commit incompatible objects on the same functional-predicate key over overlapping validity, with neither side reaching the evidence threshold. The system therefore targets RAG maintenance over corpora with extractable temporal signal and verifiable evidence; deep historical uncertainty (e.g. disputes over events with no contemporary record) and adversarial corpora (where coordinated misinformation can satisfy the evidence threshold) are out of scope and require human-in-the-loop verification, which the released annotation tool supports.

3.1 Nugget Record Model

A nugget n is defined as a tuple $(k, f, \sigma, \epsilon, \Pi)$:

- k: Nugget kind $\in$ {SemanticFact, EpisodicEvent, Instruction, UserPreference}.
- f: Atomic fact as a semantic triple (*subject*, *predicate*, *object*).
- σ: Temporal validity interval $[t_s, t_e)$ with scope, location, and source type.
- ϵ: Epistemic state with status (Active, Deprecated, Contested), rank (Preferred, Normal, Deprecated), and confidence score.

– Π: Provenance record with evidence links to source spans, creation metadata, and version lineage.

Each nugget has a stable hash-based identifier and version graph links via a parent ID. The nugget key $\kappa = (s_{\text{norm}}, p, \textit{scope})$ enables conflict detection. A nugget is retrievable at query time t if $t \in [t_s, t_e)$ and its status is Active (or Contested when uncertainty markers are desired). This differs from passage retrieval, which returns text blocks, and GraphRAG, which traverses entity graphs.

3.2 Nugget Construction Pipeline

Given a document d with timestamp t_d, the extraction process involves four stages (Figure 2). Algorithm 1 details validity inference, while Algorithm 2 outlines conflict detection.

Stage 1: Candidate Extraction. We segment the document d into sentences and process them using a sliding context window (the current sentence plus the preceding one) to preserve local context for pronoun resolution and cross-sentence references (e.g., "He founded the company"). An LLM-based extractor decomposes each window into atomic statements. Each extracted candidate consists of the text ℓ, extracted fields (*subject*, *predicate*, *object*), and character offsets representing the evidence span in the source document.

Stage 2: Canonicalization and Keying. We canonicalize extracted triples to support deduplication. Subjects are normalized through alias lookup (e.g., "USA" → "United States") and case standardization. Predicates map to a controlled vocabulary (e.g., "is the CEO of" → `chiefExecutiveOfficer`). Objects are resolved to entities where possible, or normalized to standard formats (e.g., ISO dates). The controlled vocabulary is a user-replaceable schema declaring, for each predicate, a canonical name, surface-form aliases, expected (subject, object) entity types, and a cardinality flag (functional, multi-valued, or event-log) that the conflict detector reads. To lower the adoption barrier on unfamiliar corpora, an opt-in schema-discovery module scans a representative sample, clusters surface-form predicates by frequency and entity-type co-occurrence, and proposes a starter schema that the user can edit before ingest.

We then compute the nugget key $\kappa = (s_{\text{norm}}, p, \textit{scope})$, where *scope* is derived from document metadata or textual cues. The default is `global`, while `user` and `group` denote personal and shared contexts. Excluding the object value from the key enables the identification of duplicates (same key, similar values) and conflicts (same key, different values).

3.2.1 Stage 3: Validity Inference. We assign a validity interval $\sigma_n = [t_s, t_e)$ following Algorithm 1. A temporal tagger (e.g., SUTime [10] or regex patterns) identifies date expressions and classifies them as start points, end points, or point-in-time references. The start time t_s uses an explicit start date if found (e.g., "since 2019"), otherwise defaulting to the document timestamp t_d. The end time t_e uses an explicit end date if found, otherwise defaulting to open-ended ($+\infty$).

For versioned sources such as Wikipedia, we refine intervals using revision history. If a statement in revision r_i is absent in r_{i+1}, we set t_e to the earlier of its current value and r_{i+1}'s timestamp. If a statement first appears in r_{i+1}, we set t_s to the later of its current value and r_{i+1}'s timestamp. We assume revisions reflect real-world state changes. While suitable for collaborative sources, editorial corrections risk being interpreted as temporal transitions.

Algorithm 1: Validity Interval Inference

Input: nugget n (text ℓ_n), doc d (time t_d), revision history H
Output: $\sigma_n = [t_s, t_e)$

$T \leftarrow \textsc{ExtractTemporalExpressions}(\ell_n)$
$t_s \leftarrow \begin{cases} date & \exists(_, date, \texttt{START}) \in T \\ date & \exists(_, date, \texttt{POINT}) \in T \\ t_d & \text{otherwise} \end{cases}$
$t_e \leftarrow \begin{cases} date & \exists(_, date, \texttt{END}) \in T \\ +\infty & \text{otherwise} \end{cases}$
if *d has history in H* **then**
  // tighten via revisions
  $R \leftarrow \textsc{GetRevisions}(d, H)$ // time-ordered
  for $i \leftarrow 1$ **to** $|R| - 1$ **do**
    $r_i, r_{i+1} \leftarrow R[i], R[i+1]$
    if $\ell_n \in r_i \wedge \ell_n \notin r_{i+1}$ **then**
      $t_e \leftarrow \textsc{Earlier}(t_e, \textsc{Time}(r_{i+1}))$
    if $\ell_n \notin r_i \wedge \ell_n \in r_{i+1}$ **then**
      $t_s \leftarrow \textsc{Later}(t_s, \textsc{Time}(r_{i+1}))$
$N_{\neq} \leftarrow \{\, n' \mid \kappa_{n'} = \kappa_n \wedge \text{value}(n') \neq \text{value}(n) \,\}$
foreach $n' \in N_{\neq}$ **do**
  // tighten via conflicts
  if $t_s(n') > t_s \wedge |\textsc{Evidence}(n')| \geq 2$ **then**
    $t_e \leftarrow \textsc{Earlier}(t_e, t_s(n'))$
Return $[t_s, t_e)$

Algorithm 2: Conflict Detection and State Assignment

Input: candidate nugget n_{new}, index $\mathcal{N}$, schema $\mathcal{R}$
Output: updated index and states $s_\bullet$

$\kappa \leftarrow \textsc{ComputeKey}(n_{\text{new}})$
$N_\kappa \leftarrow \{\, n \in \mathcal{N} \mid \kappa_n = \kappa \,\}$
if $N_\kappa = \emptyset$ **then**
  // new key
  $s_{n_{\text{new}}} \leftarrow$ Active; **Return** $\mathcal{N} \cup \{n_{new}\}$
foreach $n \in N_\kappa$ **do**
  // exact key match
  if $\textit{JaccardValue}(n, n_{new}) \geq 0.85$ **then**
    // deduplication
    $\pi_n \leftarrow \pi_n \cup \pi_{n_{\text{new}}}$; **Return** $\mathcal{N}$ // merge evidence
$r \leftarrow \textsc{GetRelation}(\kappa)$
$N_\cap \leftarrow \{\, n \in N_\kappa \mid \sigma_n \cap \sigma_{n_{\text{new}}} \neq \emptyset \,\}$
if $N_\cap = \emptyset$ **or** $r \in \mathcal{R}_{multi}$ **then**
  // succession / multi-valued
  $s_{n_{\text{new}}} \leftarrow$ Active; **Return** $\mathcal{N} \cup \{n_{new}\}$
foreach $n \in N_\cap$ **do**
  // functional + overlap
  if $|\pi_{n_{new}}| \geq 2 \wedge t_s(n_{new}) > t_s(n)$ **then**
    $s_n \leftarrow$ Deprecated; $t_e(n) \leftarrow t_s(n_{\text{new}})$; $s_{n_{\text{new}}} \leftarrow$ Active
  else if $|\pi_n| \geq 2 \wedge t_s(n) > t_s(n_{new})$ **then**
    $s_{n_{\text{new}}} \leftarrow$ Deprecated; $t_e(n_{\text{new}}) \leftarrow t_s(n)$
  else
    $s_n, s_{n_{\text{new}}} \leftarrow$ Contested
Return $\mathcal{N} \cup \{n_{new}\}$

Finally, we apply conflict-based refinement for functional relations. When a newer nugget with the same key κ has sufficient evidence (≥ 2 sources), we set the current nugget's t_e to the newer nugget's t_s, treating this as temporal succession. This heuristic prioritizes recency for single-valued attributes (e.g., CEO) but does not apply to multi-valued relations (e.g., board members).

3.2.2 Stage 4: Conflict Handling and State Assignment. Conflict detection is algorithmic, not generative. The LLM extracts facts from documents without cross-document awareness. The system identifies conflicts downstream when multiple nuggets share the same key κ but have different values and overlapping validity intervals.

This design provides robustness to LLM inconsistency. A hallucinated fact from a single document lacks the evidence redundancy required to deprecate existing knowledge and is isolated as Deprecated. The existing consensus remains Active until the new claim accumulates sufficient corroboration.

Following validity inference, we integrate candidates into the index (Algorithm 2) via deduplication and state assignment:

Deduplication (Merge). If the candidate matches an existing key κ, we compare object values. Exact matches or near-duplicates (Jaccard ≥ 0.85 over character n-grams) are merged. For example, "Google's CEO Sundar Pichai" and "Pichai, leader of Google" resolve to the same key and similar values, allowing storage as a single nugget with accumulated evidence.

Conflict Detection (Update). If keys match but values differ (Jaccard < 0.85) and the relation is functional, we examine temporal overlap. Non-overlapping intervals indicate succession (e.g., a CEO change), yielding a new Active nugget. Overlapping intervals trigger conflict resolution. Multi-valued relations (e.g., board members) accept concurrent values. For functional relations: if the newer candidate has ≥ 2 independent sources, the existing nugget is marked Deprecated; if the existing nugget has superior evidence, the candidate is deprecated; ambiguous cases mark both Contested, requiring ≥ 3 sources for resolution. The cardinality is a per-predicate property of the schema. Legitimate concurrency on a normally-functional role (e.g., a co-CEO arrangement during a leadership transition, or co-chairs on a board) is supported by overriding the predicate's cardinality to multi-valued in a user-supplied schema, rather than being silently misclassified as a conflict. The schema-discovery module described in Section 3 surfaces such overrides automatically when the input corpus contains multiple concurrent objects on a predicate that the default schema marks functional.

These thresholds were selected empirically, balancing noise reduction against update recall. The thresholds assume source independence; correlated errors (consistent LLM hallucinations or circular reporting) could bypass this safeguard. To mitigate this, changes to frequently accessed nuggets are flagged for human review. NuggetIndex includes an annotation tool for manual verification of extraction, normalization, validity, and conflict decisions[2].

3.3 Indexing and Retrieval

The NuggetIndex comprises four decoupled components that can be enabled or disabled based on deployment constraints:

(1) **Document Store:** A key-value store mapping nugget IDs to full records (text, validity, state, provenance).
(2) **Metadata Index:** A B-tree [5] over validity intervals and states. This component is mandatory for governance but requires negligible storage.
(3) **Sparse Index:** An inverted index storing the canonical nugget text ℓ_n for BM25 retrieval [35]. It requires no heavy inference hardware.
(4) **Dense Index:** An HNSW [28] approximate nearest neighbor structure ($M = 32$, `efConst=200`) over nugget embeddings $\mathbf{z}_n$.

This modularity supports flexible deployment. *Hybrid* mode uses all components for high-resource environments. *Lexical-Only* mode omits the dense index for resource-constrained or edge environments (e.g., browser-based search), removing embedding inference while retaining temporal governance.

3.3.1 Retrieval Algorithm. Retrieval for a query x at time t involves a four-step procedure. First, **view filtering** restricts the search to nuggets in the target view (*active* or *active-plus-contested*) that are valid at time t. The metadata index eliminates nuggets where $t \notin \sigma_n$ or where s_n is Deprecated. Second, **candidate retrieval** computes BM25 scores and retrieves approximate nearest neighbors for the query embedding within this filtered set.

Third, **score fusion** combines these components via $s(x, n) = \alpha \cdot s_{\text{BM25}}(x, n) + \beta \cdot s_{\text{dense}}(x, n)$, with hyperparameters α and β determined via grid search on development data. Finally, the system ranks nuggets by the combined score to return the top-K results.

3.3.2 Handling Contested Nuggets. Using the *active-plus-contested* view, NuggetIndex retrieves contested nuggets and explicitly flags them in the generator prompt. Unlike standard RAG, which may arbitrarily select one side of a contradiction or hallucinate a resolution, the system preserves the dispute structurally. The generator is instructed to encode this uncertainty using the following schema:

```
Established facts:
- [Active nugget 1]
- [Active nugget 2]

Disputed (sources disagree):
- [Contested nugget]: Source A says X, Source B says Y
```

This structured injection lets the generator produce nuanced answers (e.g., "while generally considered X, some sources claim Y..."). Consequently, the frequency of confident hallucinations decreases.

4 Experimental Setup

We evaluate NuggetIndex across five research questions, measuring retrieval effectiveness (RQ1–RQ2), governance accuracy (RQ3–RQ4), and practical deployment (RQ5). This section describes the evaluation framework. Section 4.2 introduces the datasets, Section 4.3 the baselines, and Section 4.4 the implementation details.

4.1 Research Questions

Five research questions guide the evaluation:

RQ1 (Coverage). Assesses whether NuggetIndex improves nugget recall and answer completeness against passage baselines.
RQ2 (Temporal Correctness). Evaluates temporal accuracy and version conflict reduction under corpus evolution.
RQ3 (Efficiency). Determines whether nugget retrieval reduces prompt length without degrading generation quality.
RQ4 (Ablations). Measures the individual contributions of granularity, validity filtering, and lifecycle states.
RQ5 (Construction Quality). Analyzes the accuracy of the automated nugget construction pipeline.

4.2 Datasets

We use four benchmarks covering static retrieval, temporal reasoning, and multi-hop question answering. To address the lack of

[2] https://github.com/searchsim-org/sigir26-nuggetindex

nugget-level annotations in standard datasets, we combine existing nuggetized resources with adapted benchmarks.

RAVine: Nuggetized MS MARCO (RQ1). For static coverage evaluation, we use RAVine [44], based on MS MARCO v2.1 [4, 39]. The dataset contains 84 queries with 3,182 gold nuggets classified as *vital* (1,602) or *okay* (1,580), linked to 2,500 documents within a corpus of approximately 10 million passages.

TimeQA (RQ2, RQ4, RQ5). For temporal correctness, we use TimeQA [12] with 12,183 questions in easy and hard splits. Derived from Wikipedia revision histories, the dataset targets entities with time-varying attributes. The hard split tests temporal disambiguation where answers vary across periods.

SituatedQA (RQ2). We use the temporal subset of SituatedQA [48] (12,227 questions). Unlike TimeQA, this dataset evaluates implicit temporal grounding without explicit time references (e.g., "current president"), testing robustness to implicit signals.

MuSiQue (RQ3). For multi-hop efficiency, we use MuSiQue [40] (47,251 questions with decomposed reasoning chains), measuring token use and latency when nugget retrieval reduces context length.

Statistical Considerations. Although RAVine contains only 84 queries, its 3,182 nuggets ensure statistical significance [15]. Multiple runs yield narrow 95% confidence intervals (±.017 to ±.034) with $p < 0.001$. The combined TimeQA and SituatedQA datasets provide over 24,000 questions for temporal evaluation.

Limitations. A primary limitation is the scarcity of nuggetized benchmarks. Standard resources like Natural Questions [21] and HotpotQA [45] lack nugget-level decomposition. Creating such annotations requires significant manual or verified automated effort. Consequently, RAVine serves as the most rigorous foundation for our coverage claims due to its expert-curated annotations.

4.3 Baselines

We compare our approach against four baseline categories to separate the effects of granularity, time awareness, and governance.

Passage Retrieval. Three standard retrievers: *BM25-Passage* (sparse lexical matching) [35]; *Dense-Passage* (DPR-style) [19]; and *Hybrid-Passage* (linear combination of BM25 and dense scores) [27].

Time-Aware Passage Retrieval. To isolate governance from simple time metadata, we augment the hybrid baseline with temporal logic. *TimeFilter* restricts retrieval to a query time window [25]. *RecencyRerank* applies score decay $s' = s \cdot \exp(-\lambda(t - t_{\text{doc}}))$ [8, 25]. *LatestSnapshot* indexes only the most recent document version [17].

Proposition Retrieval. To test granularity without governance, *Proposition-RAG* extracts propositions using the same method as NuggetIndex (Stage 1) but indexes them as plain text without validity intervals or lifecycle states [11]. *Proposition-RAG + TimeFilter* adds document-level time filtering [25].

Graph-Based Retrieval. GraphRAG [20] builds an entity graph from documents and retrieves via graph traversal and reranking using the authors' default parameters.

4.4 Implementation Details

The dense encoder uses Cohere `embed-english-v3` [13] (1024 dimensions) on the MS MARCO v2.1 corpus [29, 39]; embedding-model substitutability for retrieval is studied in [9]. We retrieve K=20 nuggets using score-level fusion with weights α=0.4 (lexical), β=0.5 (dense), and γ=0.1 (metadata). Scores are min–max normalized to $[0, 1]$ and combined as $s(d) = \alpha\, s_{\text{lex}}(d) + \beta\, s_{\text{dense}}(d) + \gamma\, s_{\text{meta}}(d)$. The time filter window is ±180 days with recency decay λ=0.001.

We use `gpt-4o-mini` [30] as the generator to reflect latency-sensitive production environments. This choice isolates performance gains attributable to governed retrieval rather than the model's parametric knowledge.

Results are means with 95% confidence intervals from five runs with distinct random seeds. Statistical significance is determined using paired bootstrap tests [38] (10,000 iterations) with Bonferroni correction [7] for multiple comparisons. The nugget store is backed by SQLite for the experiments reported here; the storage layer is a thin abstraction and a drop-in PostgreSQL/pgvector backend is supported for production deployments where multi-writer durability and external authentication are required.

5 Results

We structure the analysis by research question and keep generator settings fixed within each dataset for comparability.

5.1 RQ1: Nugget Coverage on Static QA

5.1.1 Motivation. Passage-based retrieval returns documents with mixed relevant and irrelevant content, forcing generators to extract facts from noisy context. We hypothesize that nugget-level retrieval improves ranking quality and coverage by directly indexing atomic facts and reducing redundancy through normalization.

5.1.2 Setting. We evaluate on the nuggetized RAVine benchmark [44], a static environment with over 10 million passages [39]. We compare NuggetIndex-Active against passage baselines (*BM25-Passage*, *Dense-Passage*, *Hybrid-Passage*), *Proposition-RAG*, and *GraphRAG*. NuggetIndex-Active retrieves only nuggets with valid intervals at query time and non-deprecated lifecycle state.

Passage baselines map to nuggets via evidence span containment (validated on 500 queries). Metrics include nDCG@10, R@10/20/50, Nugget Recall, and Vital Nugget Recall. Low absolute recall across all systems reflects task difficulty: retrieving documents supporting fine-grained nuggets from 10 million passages.

5.1.3 Results. Table 3 presents retrieval performance. Among passage baselines, *BM25-Passage* and *Hybrid-Passage* achieve nugget recall of .166. *Dense-Passage* yields higher nDCG@10 (.349) but lower nugget recall (.136), indicating dense representations favor semantic similarity over lexical precision for factual matching.

Proposition-RAG and GraphRAG show comparable nugget recall (.144, .142) but lower nDCG@10 (.218, .205). Heuristic decomposition introduces noise, offsetting atomic granularity benefits.

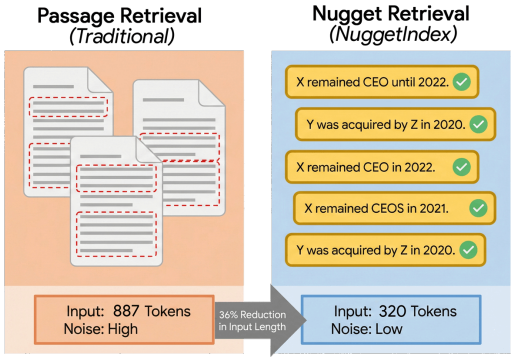
NuggetIndex-Active outperforms all baselines. Compared to *BM25-Passage*, it improves nDCG@10 by 96% (.637), R@10 by 243% (.312), nugget recall by 42% (.235), and vital nugget recall by 47% (.272). All improvements are significant ($p < 0.001$) [36].

Figure 3: Results on nuggetized MS MARCO —RAVine [44].

System	Retrieval				Nugget Coverage	
	nDCG@10	R@10	R@20	R@50	Nugget R	Vital R
Passage retrieval						
BM25-Passage	.325 ±.031	.091 ±.013	.118 ±.014	.184 ±.018	.166 ±.017	.185 ±.019
Dense-Passage	.349 ±.034	.071 ±.009	.098 ±.012	.119 ±.013	.136 ±.015	.149 ±.017
Hybrid-Passage	.324 ±.031	.091 ±.013	.118 ±.014	.184 ±.018	.166 ±.017	.185 ±.019
Proposition retrieval (atomic, no governance)						
Proposition-RAG	.218 ±.030	.062 ±.012	.100 ±.014	.100 ±.014	.144 ±.018	.158 ±.019
Graph-based retrieval						
GraphRAG	.205 ±.028	.064 ±.010	.097 ±.014	.098 ±.014	.142 ±.017	.166 ±.019
NuggetIndex						
NuggetIndex-Active	**.637**† ±.034	**.312**† ±.027	**.353**† ±.028	**.372**† ±.025	**.235**† ±.021	**.272**† ±.024

*Retrieval metrics evaluate the identification of source documents containing gold nuggets. Values represent means across 5 runs (420 queries); ± denotes 95% confidence intervals. **Bold** indicates the best result, while † marks significant improvement over the top passage baseline ($p < 0.001$).*

Figure 4: Granularity and prompt efficiency.



Passage retrieval returns long, noisy text blocks; nugget retrieval selects compact facts, reducing prompt noise and input length.

Ranking quality improves more than recall. NuggetIndex retrieves nuggets directly and maps them to documents, ensuring high relevance in top results. Normalization consolidates duplicates, and governance filters deprecated facts. NuggetIndex retrieves approximately one-third of relevant documents (R@20 .353, R@50 .372), improving over the baseline ceiling of 12%.

5.2 RQ2: Temporal Correctness Under Corpus Evolution

5.2.1 Motivation. Knowledge bases evolve over time, with facts becoming outdated or contradicted. Standard retrieval systems lack mechanisms to distinguish valid from deprecated content. We hypothesize that governed metadata—validity intervals and lifecycle states—maintains temporal correctness without sacrificing recall, overcoming the trade-off in simple time filtering.

5.2.2 Setting. We evaluate on TimeQA [12], targeting entities with time-varying attributes from Wikipedia revision histories. We compare two NuggetIndex configurations: *Active* (retrieves only *active* nuggets valid at query time) and *Full* (i.e., *active-plus-contested*, includes contested nuggets with uncertainty prompting). Baselines include *Hybrid-Passage* and three time-aware variants: *TimeFilter* (±180 days), *RecencyRerank* (λ=0.001), and *LatestSnapshot*. We also include Proposition-RAG (with/without filtering) and GraphRAG.

Metrics include Temporal Correctness (TC, fraction of nuggets valid at query time), Conflict Rate (fraction of answers with inconsistent facts), Nugget R@20, and generation-level Recall/F1 via autograder [34] (89% human agreement).

5.2.3 Results. Table 5 reveals a core trade-off: time-aware filtering improves correctness but collapses recall. *Hybrid-Passage* achieves .840 TC with .497 recall; *TimeFilter* improves TC to .921 but drops recall to .002. *LatestSnapshot* yields lowest conflict rate (.001) but zero recall. Proposition-RAG reduces conflict rates by 37% (.161 to .102) via atomic decomposition, but TC remains at .838—granularity alone is insufficient. GraphRAG achieves low conflicts (.045) but lower recall (.374).

NuggetIndex-Active resolves this trade-off, reaching .931 TC at .343 recall (+9.1pp TC over *Hybrid-Passage* with no recall loss) and cutting conflicts by 55% to .072. NuggetIndex-Full achieves .934 TC with slightly higher conflicts (.112), providing historical context. Handling "*contested*" states is key. By flagging conflicting updates as disputed rather than forcing binary decisions, the system allows generators to qualify answers (e.g., "The exact date is disputed..."), turning potential errors into correct uncertainty statements.

SituatedQA Evaluation. On SituatedQA [48], NuggetIndex-Active achieves .010 conflict rate (96% reduction vs. *Hybrid-Passage* at .230) while preserving .700 recall, confirming governance generalizes beyond revision-based corpora.[3]

5.3 RQ3: Efficiency and Token Economy

5.3.1 Motivation. Large language models costs scale with context length, making prompt efficiency critical for production. Passage retrieval includes redundant content, inflating token counts. We hypothesize that nugget-level retrieval reduces input length while maintaining generation quality, enabling deployment in latency-sensitive or resource-constrained environments.

5.3.2 Setting. We evaluate on TimeQA [12] (1,000 queries) and MuSiQue [40] (2,154 queries). Metrics include build time, build cost (GPT-4o-mini pricing [31]), index size, retrieval latency (P50/P95), median input length, and end-to-end P50 latency.

5.3.3 Results. Table 2 and Figure 4 shows efficiency results. Passage systems require median 887 tokens with negligible build cost. Proposition-RAG reduces to 346 tokens ($1.40 build cost); GraphRAG achieves 54 tokens through aggressive compression that may lose context. NuggetIndex requires 320 tokens—64% reduction over passage retrieval—with higher information density than propositions.

NuggetIndex matches Passage retrieval latency (0.8ms P50 Active; 1.2ms Full vs. 1–2ms). End-to-end latency (1,269ms P50) equals

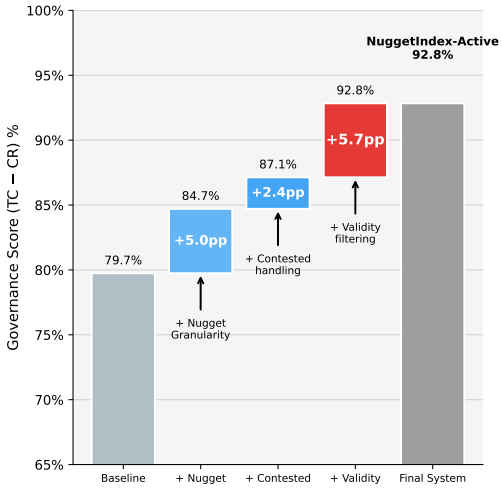
[3] SituatedQA: https://github.com/searchsim-org/sigir26-nuggetindex/situatedqa

Figure 5: Results on TimeQA [12].

System	Nugget R@20	Nugget R (gen)	Nugget F1 (gen)	Temporal Correctness	Conflict Rate
Passage retrieval					
Hybrid-Passage	**.497**±.019	.143±.007	.165±.007	.840±.006	.161±.008
Time-aware passage retrieval					
Hybrid + TimeFilter	.002±.002	.111±.006	.121±.006	.921±.005	.148±.008
Hybrid + RecencyRerank	.002±.002	.110±.006	.121±.007	.921±.005	.148±.008
Hybrid + LatestSnapshot	.000±.000	.059±.009	.059±.009	.925±.009	**.001**±.001
Proposition retrieval (atomic, no governance)					
Proposition-RAG	.492±.019	.128±.008	.140±.008	.838±.006	.102±.009
Proposition + TimeFilter	.002±.002	.101±.007	.106±.007	.930±.004	.114±.011
Graph-based retrieval					
GraphRAG	.374±.018	**.271**±.015	**.277**±.015	.846±.009	.045±.007
NuggetIndex					
NuggetIndex-Active	.343±.018	.115±.007	.125±.007	.931±.005	.072±.008
NuggetIndex-Full	.359±.019	.115±.007	.125±.008	**.934**±.004	.112±.010

Temporal Correctness (TC) is the fraction of covered nuggets valid at query time. Conflict Rate is the fraction of answers containing temporally inconsistent facts (lower is better). ***Bold*** *indicates the best result.*

Figure 6: Contribution to Governance Score.



Governance Score = Temporal Correctness - Conflict Rate; higher scores indicate greater temporal accuracy and fewer contradictions.

Table 2: Efficiency comparison on TimeQA (1,000 queries).

System	Build Time (h)	Build Cost ($)	Index Size (GB)	Retrieval P50/P95	Input Length	E2E P50
BM25-Passage	.001	0	.016	2/5	887	1,269
Hybrid-Passage	.002	.04	.014	1/3	887	1,283
Proposition-RAG	.004	1.40	.014	1/4	346	1,274
GraphRAG	.007	1.30	.003	0/1	**54**	**1,032**
NuggetIndex-Active	.002	1.32	.009	1/3	320	1,269
NuggetIndex-Full	.003	1.32	.009	1/4	320	1,305

Table 3: Ablation study on TimeQA [12] (500 queries).

Configuration	Nugget R@20	ΔR	Governance Score†	ΔGov
NuggetIndex-Active	.343	—	**.928**	—
− validity filtering	.461	+11.8	.833	−9.5
− lifecycle states	.343	—	.928	—
− contested handling	.352	+0.9	.888	−4.0
− normalization	.341	−0.2	.928	—
− nugget granularity	.378	+3.5	.845	−8.3
− all governance	.482	+13.9	.797	−13.1

† *Governance Score = Temporal Correctness − Conflict Rate.*

BM25-Passage, as reduced generation offsets retrieval overhead. The index for 56,990 nuggets requires only 9.3MB, enabling resource-constrained deployment with sub-millisecond latency.

MuSiQue Evaluation. On MuSiQue [40], NuggetIndex-Active reduces tokens by 82% (339 vs. 1,884), confirming that nugget-level retrieval maintains its advantage on multi-hop reasoning.[4]

5.4 RQ4: Ablation Studies

5.4.1 Motivation. NuggetIndex comprises: nugget granularity, validity filtering, lifecycle states, contested handling, and normalization. Understanding individual contributions guides design decisions and identifies which components justify their complexity. We quantify the trade-off between retrieval coverage and temporal integrity introduced by each governance mechanism.

5.4.2 Setting. We analyze 500 queries from TimeQA [12], prioritizing passages with Deprecated or Contested nuggets. We use NuggetIndex with BM25 retrieval to isolate governance effects.

We define *Governance Score* as Temporal Correctness minus Conflict Rate (1.0 = perfect). Ablations disable individual components: validity filtering (retrieving all nuggets regardless of interval), lifecycle states (treating all nuggets as Active), contested handling (treating disputed nuggets as Active without uncertainty markers), normalization, and nugget granularity (replacing with passage-level retrieval). We also remove all governance components.

5.4.3 Results. Table 3 and Figure 6 present component contributions. Removing *validity filtering* causes the largest shift: recall increases by 11.8 percentage points while Governance Score decreases by 9.5pp (.928 to .833), indicating the system prioritizes factual currency over coverage. *Lifecycle states* show no measurable impact—TimeQA contains few deprecated facts (84 of 69,903), limiting evaluation of this feature. Removing *contested handling* decreases Governance Score by 4.0pp and increases recall by 0.9pp, showing this component effectively filters contradictory information. Disabling *normalization* does not affect Governance Score.

Replacing nuggets with passages increases recall by 3.5pp but drops Governance Score by 8.3pp. Removing all governance yields

[4] MuSiQue: https://github.com/searchsim-org/sigir26-nuggetindex/musique

Table 4: Retrieval mode ablation on TimeQA ($N = 200$).

Mode	R@10	nDCG	Temporal Correctness	Latency
Hybrid (Default)	**.305**	**.102**	1.00	12.3ms
Semantic-Only	.260	.083	1.00	11.5ms
Lexical-Only	.282	.092	1.00	**0.4ms**
Ablation: No Temporal Filter (High Recall / Low Correctness)				
Lexical-Only	**.372**	.109	.899	**0.3ms**
Hybrid	.357	**.110**	.880	12.3ms

***Bold** indicates best in class. Lexical-Only removes embedding costs entirely.*

+13.9pp recall but −13.1pp Governance Score (to .797). Overall, the system trades coverage for temporal accuracy.

Retrieval Mode Ablation. While the previous ablation examines governance components, we also analyze retrieval architecture by decoupling sparse and dense retrieval (Table 4). *Hybrid* achieves 30.5% recall with perfect temporal correctness. Notably, **sparse retrieval outperforms dense** (.282 vs. .260 recall), suggesting that for atomic nuggets, precise lexical matching of entities and relations is more effective than broad semantic similarity from embeddings. The *Lexical-Only* mode emerges as a highly efficient deployment option, retaining 92% of hybrid recall while reducing P50 latency by a factor of 30 (12.3ms to 0.4ms), confirming governed retrieval is feasible for edge environments without embedding inference.

5.5 RQ5: Construction Quality

5.5.1 Motivation. NuggetIndex effectiveness depends on the quality of its automated construction pipeline. Errors in extraction, normalization, validity inference, or conflict detection propagate to retrieval and generation. Evaluating each stage establishes confidence in the system's foundations.

5.5.2 Setting. We evaluate using 500 samples with ground truth and manually annotated data from TimeQA [12].

Extraction: 500 nuggets from 124 passages, stratified by property type. Two annotators independently identified atomic facts (blind to system output) then evaluated whether each extracted nugget was correct and atomic. Metrics: (fraction of extracted nuggets that are valid facts), Recall (fraction of identified facts extracted), and Atomicity (fraction of nuggets containing exactly one fact). Inter-annotator agreement: κ=0.69 (substantial [23]).

Normalization: 500 entity pairs measuring Alias Resolution Recall (ability to link equivalent mentions) and False Merge Rate (distinct facts incorrectly merged). Inter-annotator: κ=0.97.

Validity inference (Algorithm 1): TimeQA nuggets with ground-truth temporal validity windows. Metrics: Start Time Accuracy (±30 days), End Time Detection Recall (ability to identify when facts end), End Time Accuracy (precision when detected).

Conflict detection (Algorithm 2): state-change decisions on subject-predicate keys (κ=0.73, substantial). Metrics: Deprecation Precision, Contestation Appropriateness, Missed Conflict Rate.

We release the manually annotated data to support research on nugget extraction and conflict detection evaluation.[5]

[5]Annotation: https://github.com/searchsim-org/sigir26-nuggetindex/annotation_tool

Table 5: Construction pipeline quality.

Stage	Metric	Value
Extraction	Precision	.860
	Atomicity	.943
Normalization	Alias resolution recall	.600
	False merge rate	.016
Validity inference	Start time accuracy	.868
	End time detection recall	.667
	End time accuracy	.955
Conflict detection	Deprecation precision	1.000
	Contestation appropriateness	1.000
	Missed conflict rate	.000

Inter-annotator agreement: extraction atomicity κ=.69, normalization κ=.97. Higher is better for all metrics except false merge rate and missed conflict rate.

5.5.3 Results. Table 5 presents results. **Extraction** achieves .860 precision and .943 atomicity, indicating most nuggets contain a single fact. **Normalization** yields .600 alias recall with .016 false merge rate, prioritizing precision to prevent incorrect conflict detection.

Validity inference identifies start times well (.868); end times are accurate when found (.955) but the recall is lower (.667) due to facts ending without textual cues—revision-based inference mitigates this vs. text-only methods. **Conflict detection** achieves perfect deprecation precision 1.000 with no missed conflicts.

We quantified error propagation through an oracle experiment on 500 queries by replacing extracted nuggets with human-verified data. This improved recall by 3.7 percentage points and temporal correctness by 4.5 percentage points, demonstrating that the system remains effective despite extraction imperfections.

5.6 Multi-Hop Reasoning

5.6.1 Motivation. Multi-hop questions require aggregating information across multiple documents, challenging retrieval systems to identify and chain relevant facts. While passage-based retrieval may capture broad context, nugget retrieval surfaces atomic facts as reasoning steps. We evaluate whether NuggetIndex supports multi-hop reasoning and how performance scales with corpus size.

5.6.2 Setting. We evaluate on HotpotQA [45] in two settings: *closed-corpus* (query-specific contexts, ∼1,000 passages) and *pooled-corpus* (500 queries combined, ∼5,000 passages) to simulate open-domain retrieval. Metrics: Intermediate Recall (coverage of all supporting passages required for reasoning), Final Recall (coverage of the answer-bearing passage), Success Rate (fraction of queries where the system retrieves sufficient context). We compare Hybrid-Passage, Proposition-RAG, GraphRAG, and NuggetIndex-Active.

5.6.3 Results. Table 6 presents results. In *closed-corpus*, Hybrid-Passage achieves .890 intermediate recall. Proposition-RAG drops to .800 as sentence-level splitting limits context, while GraphRAG underperforms at .690 due to entity-centric extraction missing relational facts. NuggetIndex-Active remains competitive at .850, validating atomic fact aggregation effectiveness.

In *pooled-corpus*, recall drops for all systems as the search space grows, but NuggetIndex-Active is most robust, reaching .790 intermediate recall vs. .775 (Proposition-RAG) and .645 (GraphRAG).

Table 6: Multi-hop QA results on HotpotQA [45].

	Closed-corpus (1K)			Pooled-corpus (5K)		
System	Inter. R	Final R	Success	Inter. R	Final R	Success
Hybrid-Passage	**.890**	**.805**	**1.000**	**.855**	**.775**	**.990**
Proposition-RAG	.800	.730	.970	.775	.690	.970
GraphRAG	.690	.555	.930	.645	.480	.920
NuggetIndex-Active	.850	.775	.990	.790	.715	.980

Intermediate recall (Inter. R) measures coverage of all supporting passages. Final recall (Final R) measures coverage of the answer-bearing passage.

Hybrid-Passage outperforms NuggetIndex-Active by 1.5–4.0pp on this static benchmark. HotpotQA passages are human-curated paragraphs that co-locate the multi-hop evidence, so lexical matching alone recovers most of them in one pass. The corpus is also static, with no temporal labels, no rename events, and no functional-predicate conflicts. Validity filtering and contestation cannot help when there is nothing to filter. NuggetIndex therefore pays the cost of decomposition without the benefits of governance. We expect the gap to invert on a dynamic variant of HotpotQA in which a fraction of supporting passages is replaced by stale duplicates with revised source dates. The temporal-correctness gains in Section 5.2 (.931 vs .840) and the conflict-detection precision in Section 5.5 are the complementary best case for the governance signals.

5.7 Summary of Findings

The results yield five main conclusions. First, nugget-level retrieval outperforms Passage baselines, improving nugget recall by 42% and vital nugget recall by 47% (RQ1). Second, governed metadata increases temporal correctness by 9.1 percentage points (.931 vs .840) and reduces conflict rates by 55% compared to strong Passage baselines, avoiding the recall loss typical of time-filtered methods (RQ2). Third, the compact nugget format reduces generator input by 64% (median 887 to 320 tokens), while the 9.3MB index supports sub-millisecond latency suitable for resource-constrained environments (RQ3). Fourth, validity filtering is the primary driver of temporal accuracy, and sparse retrieval outperforms dense methods on atomic nuggets, enabling edge deployment without embedding inference (RQ4). Fifth, the automated pipeline demonstrates .860 extraction precision and .943 atomicity; oracle experiments indicate that imperfect governance remains effective, as human verification yields only marginal improvements of 3.7–4.5 percentage points (RQ5).

6 Discussion and Limitations

The performance gains stem from separating fact validity from document timestamps. Standard time-filtering operates at document level, forcing a trade-off between correctness and recall. NuggetIndex resolves this by governing individual facts, filtering deprecated content while preserving valid historical information.

Structurally, NuggetIndex balances unstructured retrieval with formal knowledge bases. It avoids rigid ontologies like Wikidata while offering greater transparency than graph-based methods. The modular index architecture enables deployment flexibility: hybrid mode for maximum recall, lexical-only for edge environments.

For corpora whose freshest source predates the query time by more than a configurable threshold (default: one year), an optional freshness-fallback layer issues a single web-search query against an external provider (Tavily, Serper, Exa, or Brave) [46] and ingests the returned snippets through the same extractor and conflict pipeline; the fallback fires only when the existing store has no Active fact valid at the query time, and the retrieved nuggets are tagged with a distinct provenance type so downstream filters can require corroboration before promoting a fallback claim to Active.

Cost and reproducibility. The construction pipeline relies on an LLM extractor, and the cost is one-time and amortized. Table 2 reports offline build cost on TimeQA at $1.32 with `gpt-4o-mini` [30], comparable to Proposition-RAG ($1.40) and GraphRAG ($1.30). At runtime, NuggetIndex retrieves in 0.8 ms (P50) and reduces the median generator input by 64% (887 to 320 tokens, RQ3). On a 47K-query workload such as MuSiQue, this saving dwarfs the build cost within hours of serving. The dependency on `gpt-4o-mini` is at the extractor boundary only. The retrieval and governance layers are pure Python over SQLite, and adopters can swap in any OpenAI-compatible endpoint (open-weights or self-hosted) via the released `LLMConfig` without changing the pipeline.

Limitations. Validity inference and entity normalization remain primary weaknesses. End time detection achieves only 0.667 recall when facts expire without textual cues. Normalization errors cause missed conflicts when distinct mentions resolve to different keys. The Jaccard-based deduplication (≥ 0.85) may miss semantic equivalence in complex phrasing, and exact key matching fails when normalization errors occur. We plan to incorporate embedding-based deduplication, deeper knowledge base linking for entity resolution, improved end-time detection via cross-document signals, and streaming updates for real-time corpus maintenance.

7 Conclusion

This work addresses the mismatch between nugget-based evaluation and static retrieval units in RAG systems. We introduce NuggetIndex, a retrieval index where atomic facts are stored as managed records with validity intervals, lifecycle states, and provenance. Filtering by these properties prevents propagation of outdated or conflicting information.

Experiments across four benchmarks demonstrate 42% improvement in nugget recall, 9.1pp increase in temporal correctness, and 55% reduction in conflict rates—without the recall collapse of time-filtered baselines. The compact format reduces generator input by 64% and yields lightweight indices (9.3MB for 56,990 nuggets) suitable for edge deployment. Oracle experiments confirm that even imperfect automated governance provides most of the benefit, with human verification yielding only marginal gains.

Acknowledgments

This work has received funding from the European Union's Horizon Europe research and innovation program under grant agreement No. 101070014 (OpenWebSearch.EU), and the Bavarian State Ministry of Economic Affairs, Regional Development, and Energy (StMWi).

References

[1] [n. d.]. Wikidata: Data model. https://www.wikidata.org/wiki/Wikidata:Data_model. Accessed: 2025-12-08.

[2] 2013. *SPARQL 1.1 Query Language*. W3C Recommendation. World Wide Web Consortium (W3C). https://www.w3.org/TR/sparql11-query/

[3] Amazon Web Services. 2024. Introducing the GraphRAG Toolkit: Lexical Graph for Amazon Neptune. AWS Database Blog. https://aws.amazon.com/blogs/database/introducing-the-graphrag-toolkit/

[4] Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, et al. 2016. Ms marco: A human generated machine reading comprehension dataset. *arXiv preprint arXiv:1611.09268* (2016).

[5] Rudolf Bayer and Edward McCreight. 1970. Organization and maintenance of large ordered indices. In *Proceedings of the 1970 ACM SIGFIDET (Now SIGMOD) Workshop on Data Description, Access and Control*. 107–141.

[6] Klaus Berberich, Srikanta Bedathur, Omar Alonso, and Gerhard Weikum. 2010. A language modeling approach for temporal information needs. In *European conference on information retrieval*. Springer, 13–25.

[7] Carlo Bonferroni. 1936. Teoria statistica delle classi e calcolo delle probabilita. *Pubblicazioni del R istituto superiore di scienze economiche e commericiali di firenze* 8 (1936), 3–62.

[8] Ricardo Campos, Gaël Dias, Alípio Jorge, and Célia Nunes. 2016. GTE-Rank: A time-aware search engine to answer time-sensitive queries. *Information Processing & Management* 52, 2 (2016), 273–298.

[9] Laura Caspari, Kanishka Ghosh Dastidar, Saber Zerhoudi, Jelena Mitrovic, and Michael Granitzer. 2024. Beyond benchmarks: Evaluating embedding model similarity for retrieval augmented generation systems. *arXiv preprint arXiv:2407.08275* (2024).

[10] Angel X Chang and Christopher D Manning. 2012. Sutime: A library for recognizing and normalizing time expressions.. In *Lrec*, Vol. 12. 3735–3740.

[11] Tong Chen, Hongwei Wang, Sihao Chen, Wenhao Yu, Kaixin Ma, Xinran Zhao, Hongming Zhang, and Dong Yu. 2024. Dense x retrieval: What retrieval granularity should we use?. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 15159–15177.

[12] Wenhu Chen, Xinyi Wang, and William Yang Wang. 2021. A dataset for answering time-sensitive questions. *arXiv preprint arXiv:2108.06314* (2021).

[13] Cohere. [n. d.]. An Overview of Cohere's Models (embed-english-v3.0). https://docs.cohere.com/docs/models. Accessed: 2026-01-19.

[14] Yimin Deng, Yuxia Wu, Yejing Wang, Guoshuai Zhao, Li Zhu, Qidong Liu, Derong Xu, Zichuan Fu, Xian Wu, Yefeng Zheng, et al. 2025. A Multi-Expert Structural-Semantic Hybrid Framework for Unveiling Historical Patterns in Temporal Knowledge Graphs. In *Findings of the Association for Computational Linguistics: ACL 2025*. 20553–20565.

[15] Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. 2018. The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 1383–1392.

[16] Shengbo Gong, Xianfeng Tang, Carl Yang, et al. 2025. Beyond Chunks and Graphs: Retrieval-Augmented Generation through Triplet-Driven Thinking. *arXiv preprint arXiv:2508.02435* (2025).

[17] Daniel Huwiler, Kurt Stockinger, and Jonathan Fürst. 2025. VersionRAG: Version-Aware Retrieval-Augmented Generation for Evolving Documents. *arXiv preprint arXiv:2510.08109* (2025).

[18] Gautier Izacard and Edouard Grave. 2021. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th conference of the european chapter of the association for computational linguistics: main volume*. 874–880.

[19] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick SH Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering.. In *EMNLP (1)*. 6769–6781.

[20] Simon Knollmeyer, Oğuz Caymazer, and Daniel Grossmann. 2025. Document GraphRAG: Knowledge Graph Enhanced Retrieval Augmented Generation for Document Question Answering Within the Manufacturing Domain. *Electronics* 14, 11 (2025), 2102.

[21] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. 2019. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics* 7 (2019), 453–466.

[22] Weronika Łajewska and Krisztian Balog. 2025. Ginger: Grounded information nugget-based generation of responses. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2723–2727.

[23] GG Landis JRKoch. 1977. The measurement of observer agreement for categorical data. *Biometrics* 33, 1 (1977), 159174.

[24] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.

[25] Xiaoyan Li and W. Bruce Croft. 2003. Time-Based Language Models. In *Proceedings of the 12th International Conference on Information and Knowledge Management (CIKM '03)*. 469–475.

[26] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. Lost in the middle: How language models use long contexts, 2023. *URL https://arxiv. org/abs/2307.03172* (2023).

[27] Xueguang Ma, Kai Sun, Ronak Pradeep, and Jimmy Lin. 2021. A replication study of dense passage retriever. *arXiv preprint arXiv:2104.05740* (2021).

[28] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence* 42, 4 (2018), 824–836.

[29] MS MARCO Team. [n. d.]. MS MARCO: A Collection of Datasets Focused on Deep Learning in Search. https://microsoft.github.io/msmarco/. Accessed: 2026-01-19.

[30] OpenAI. [n. d.]. GPT-4o mini model — OpenAI API documentation. https://platform.openai.com/docs/models/gpt-4o-mini. Accessed: 2026-01-19.

[31] OpenAI. 2026. Pricing. https://platform.openai.com/docs/pricing. Accessed 2026-01-20.

[32] Andrew Parry, Maik Fröbe, Harrisen Scells, Ferdinand Schlatt, Guglielmo Faggioli, Saber Zerhoudi, Sean MacAvaney, and Eugene Yang. 2025. Variations in relevance judgments and the shelf life of test collections. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 3387–3397.

[33] Virgil Pavlu, Shahzad Rajput, Peter B. Golbus, and Javed A. Aslam. 2012. IR System Evaluation Using Nugget-Based Test Collections. In *Proceedings of the Fifth ACM International Conference on Web Search and Data Mining (WSDM '12)*. ACM, 393–402.

[34] Ronak Pradeep, Nandan Thakur, Shivani Upadhyay, Daniel Campos, Nick Craswell, and Jimmy Lin. 2024. Initial Nugget Evaluation Results for the TREC 2024 RAG Track with the AutoNuggetizer Framework. *CoRR* abs/2411.09607 (2024). https://arxiv.org/abs/2411.09607

[35] Stephen Robertson and Hugo Zaragoza. 2009. *The probabilistic relevance framework: BM25 and beyond*. Vol. 4. Now Publishers Inc.

[36] Mark D Smucker, James Allan, and Ben Carterette. 2007. A comparison of statistical significance tests for information retrieval evaluation. In *Proceedings of the sixteenth ACM conference on Conference on information and knowledge management*. 623–632.

[37] Richard T Snodgrass. 2012. *The TSQL2 temporal query language*. Vol. 330. Springer Science & Business Media.

[38] Robert J Tibshirani and Bradley Efron. 1993. An introduction to the bootstrap. *Monographs on statistics and applied probability* 57, 1 (1993), 1–436.

[39] TREC RAG Organizers. 2024. TREC 2024 RAG Corpus: MS MARCO V2.1 Document Corpus and Segmented Version. Blog post. https://trec-rag.github.io/annoucements/2024-corpus-finalization/ Accessed: 2026-01-19.

[40] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multihop Questions via Single-hop Question Composition. *Transactions of the Association for Computational Linguistics* 10 (2022), 539–554.

[41] Ellen M Voorhees and L Buckland. 2003. Overview of the TREC 2003 Question Answering Track.. In *TREC*, Vol. 2003. 54–68.

[42] Ellen M. Voorhees and Lori Buckland. 2003. Overview of the TREC 2003 Question Answering Track. In *Proceedings of the Twelfth Text REtrieval Conference (TREC 2003)*. 54–68.

[43] Denny Vrandečić and Markus Krötzsch. 2014. Wikidata: a free collaborative knowledgebase. *Commun. ACM* 57, 10 (2014), 78–85.

[44] Yilong Xu, Xiang Long, Zhi Zheng, and Jinhua Gao. 2025. Ravine: Reality-aligned evaluation for agentic search. *arXiv preprint arXiv:2507.16725* (2025).

[45] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing*. 2369–2380.

[46] Saber Zerhoudi, Michael Dinzinger, Michael Granitzer, and Jelena Mitrovic. 2026. OwlerLite: Scope-and Freshness-Aware Web Retrieval for LLM Assistants. *arXiv preprint arXiv:2601.17824* (2026).

[47] Saber Zerhoudi and Michael Granitzer. 2024. Personarag: Enhancing retrieval-augmented generation systems with user-centric agents. *arXiv preprint arXiv:2407.09394* (2024).

[48] Michael Zhang and Eunsol Choi. 2021. SituatedQA: Incorporating extra-linguistic contexts into QA. In *Proceedings of the 2021 conference on empirical methods in natural language processing*. 7371–7387.